Solving Hanabi with Monte Carlo Tree Search

Kaan Uçar
Supervisor: Giulio Romanelli
Swiss Data Center, EPFL, Switzerland

December 15, 2024

https://github.com/Kaan-wq/mcts_hanabi

Abstract

This report explores the optimization and evaluation of Monte Carlo Tree Search (MCTS) approaches for the cooperative card game Hanabi. We first conduct a systematic performance analysis of existing Hanabi environments, demonstrating that the DeepMind implementation executes actions approximately 100 times faster than alternatives. Through algorithmic optimizations and parallelization techniques, we significantly improve the efficiency of RIS-MCTS (Re-determinizing Information Set MCTS), reducing the execution time for 100 episodes from 43 minutes to 2 minutes while maintaining the same number of rollouts.

Our evaluation of RIS-MCTS reveals a discrepancy of approximately one point between the implementation and published results, despite extensive parameter exploration. Additionally, we present the first attempt to apply AlphaZero methodology to Hanabi, providing detailed analysis of the challenges encountered in adapting this approach to a cooperative, partially observable game setting.

This work contributes both practical improvements in Hanabi environment performance and critical insights into the applicability of state-of-the-art search methods in cooperative games with partial information.

Contents

1 Introduction

2 Related Work
2.1 Information Set Monte Carlo Tree Search . . . . . .
2.2 Re-determinizing Information Set Monte Carlo Tree Search in Hanabi . .
2.3 Improving Policies via Search in Cooperative Partially Observable Games
2.4 How to Make the Perfect Fireworks Display: Two Strategies for Hanabi .

3 Methods
3.1 Environment . . . . . .
3.2 MCTS Baseline . . . . . .
3.3 Optimizations . . . . . .
3.4 AlphaZero . . . . . .

4 Results
4.1 MCTS Baseline Evaluation . . . . . .
4.2 AlphaZero . . . . . .

5 Conclusion

6 Annexes
6.1 AlphaZero Policy Observations . . . . . .
6.2 Rules Reducing the Branching Factor of MCTS Baseline . . . . . .
6.3 Environment Details . . . . . .
6.4 Rules of Hanabi . . . . . .

Introduction

The field of artificial intelligence has witnessed remarkable achievements in recent years, particularly in game-playing agents. From IBM's Deep Blue defeating Garry Kasparov in chess to DeepMind's AlphaGo mastering the ancient game of Go, these milestones have consistently pushed the boundaries of what machines can achieve [Silver et al., 2017]. However, these successes have predominantly focused on competitive, perfect information games where agents have complete knowledge of the game state. The next frontier lies in developing AI systems capable of handling partial information and, more importantly, cooperation - challenges that more closely mirror real-world scenarios where agents must make decisions with incomplete information while coordinating with others.

Hanabi, a cooperative card game that won the prestigious Spiel des Jahres award in 2013, presents a fascinating challenge in this space [Baffier et al., 2017]. Players must work together to build firework stacks of five different colors, but with a crucial twist: players can see everyone's cards except their own. This information asymmetry, combined with limited communication opportunities, forces players to reason about their teammates' knowledge and intentions - a fundamental aspect of cooperative intelligence. The game has attracted significant attention from the AI research community [Bard et al., 2020], serving as a benchmark for testing algorithms' ability to handle both partial observability and implicit communication.

Monte Carlo Tree Search (MCTS) has emerged as a particularly promising approach for Hanabi, given its success in perfect information games. However, applying MCTS to partially observable cooperative games presents unique challenges that require innovative solutions [Cowling et al., 2012]. The current state-of-the-art, RIS-MCTS (Redeterminizing Information Set Monte Carlo Tree Search), specifically addresses these challenges through dynamic re-sampling of game states [Goodman, 2019].

This report focuses on three main objectives. First, we conduct a comprehensive comparison of existing Hanabi environments to identify the most efficient implementation for MCTS rollouts, addressing a crucial practical concern in the application of search-based methods. Second, we optimize the RIS-MCTS implementation, significantly improving its computational efficiency while maintaining its theoretical guarantees. Finally, we explore the novel application of AlphaZero methodology to Hanabi, investigating its potential in cooperative partially observable games - an approach that has not been previously attempted.

Through these investigations, we aim to advance our understanding of search-based methods in cooperative games and provide practical improvements to existing algorithms. Our work not only contributes to the specific domain of Hanabi but also offers insights into the broader challenge of developing AI systems capable of sophisticated cooperation under uncertainty - a crucial step toward more general artificial intelligence.

Related Work

MCTS is originally a technique designed for deterministic zero-sum with perfect information games. Hanabi is a stochastic cooperative game with incomplete information, so it is not trivial to apply MCTS in this context. One popular technique to apply MCTS to hidden information games is to sample a possible state of the game and then proceed as in a perfect information game. After a few rollouts, the results are aggregated and one single answer should be given by the algorithm.

However, this approach comes with a few problems :

1. **Non-Locality:** A phenomenon where the optimal decision in a hidden information game depends on understanding the full context of players' beliefs and rational behaviors, not just the immediate game state. Certain states that appear possible when viewed in isolation become impossible when considering rational player behavior.

   Example: Consider a 2-player Hanabi situation where only R4 and R5 remain to be played, and your partner gives you a number 4 hint. Given that your partner can see all your cards, this hint carries deeper meaning beyond just identifying a 4. If the hinted 4 were any color except red, the hint would be useless since only red cards are needed. Therefore, while a determinization might consider non-red 4s as possible states, these states are actually impossible when considering rational player behavior - your partner would never waste a hint on an unplayable 4. This illustrates non-locality: the meaning of your partner's hint cannot be evaluated in isolation, but must be interpreted within the broader context of rational gameplay strategy and shared knowledge.

2. **Strategy Fusion:** A fundamental problem in hidden information games where an algorithm thinks it can make different decisions based on different determinizations, when in reality a player must choose a single action that works for all indistinguishable states.

   Example: Consider a simplified Hanabi situation where you have exactly one card that you know is either R4 or R5, and R4 must be played before R5. Playing the wrong card loses the game. When sampling possible states, in determinization 1 where your card is R4, the algorithm says "Play it!", while in determinization 2 where your card is R5, the algorithm says "Don't play it!". The problem is that the algorithm suggests different actions based on information you don't actually have in the real game. In reality, you have to choose ONE action without knowing which card you have. This is strategy fusion - the algorithm incorrectly "fuses" strategies from different determinizations where it could see the hidden information, leading to unrealistic evaluations.

2.1 Information Set Monte Carlo Tree Search

Information Set Monte Carlo Tree Search (IS-MCTS) [Cowling et al., 2012] represents a fundamental shift from traditional MCTS by using information sets as nodes instead of

game states. This approach naturally aligns with how players actually perceive the game - through partial information rather than complete states.

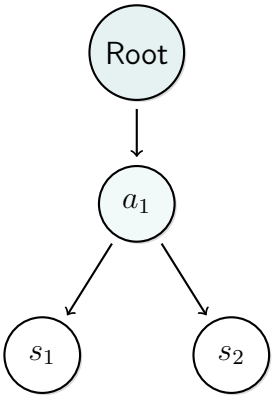

Figure 2.1: Information Set Tree: Represents all possible states in each set

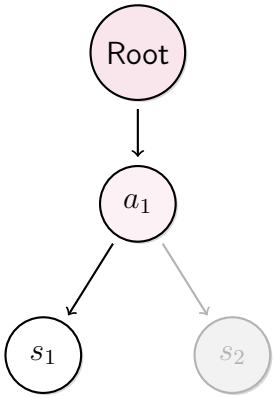

Figure 2.2: Determinization Tree: Samples one possible state

Single Observer IS-MCTS (SO-ISMCTS)

SO-ISMCTS [Cowling et al., 2012] maintains a single tree where nodes represent information sets from the root player's perspective. For each iteration, it samples one possible state at the root consistent with the current information set. During simulation, other players make random legal moves within the constraints of the sampled game state.

This approach naturally resolves the strategy fusion problem since decisions are made at the information set level, exactly matching the player's actual decision-making context. The algorithm doesn't need to aggregate results across different determinizations as they are automatically grouped in their respective information sets. However, SO-ISMCTS does not address the non-locality issue, as it still cannot capture the complex interdependencies between players' beliefs and rational actions.

Multiple Observer IS-MCTS (MO-ISMCTS)

MO-ISMCTS [Cowling et al., 2012] constructs separate trees for each player, with decisions made from the perspective of the root player's sampled state. Theoretically, this approach could address non-locality since each player maintains their own tree and makes decisions consistently within a given determinization. The separate trees could capture broader strategic patterns as each player knows their own cards and can develop strategies through their individual trees, potentially accounting for the interconnected nature of decisions.

However, this approach fails specifically in Hanabi due to the game's unique "anti-information" structure where players cannot see their own cards. During a rollout in player A's tree, when we sample a possible state and choose an action (such as giving a rank 1 hint to player B), we encounter a fundamental problem: if player B has a playable W5, the algorithm will always give a positive reward for playing it because the tree assumes player B knows their cards. This contradicts Hanabi's core mechanic and leads to unrealistic evaluations. Thus, while MO-ISMCTS could theoretically capture non-locality in standard hidden information games, it breaks down in Hanabi's specific information structure.

2.2 Re-determinizing Information Set Monte Carlo Tree Search in Hanabi

RIS-MCTS [Goodman, 2019] extends MO-ISMCTS by introducing dynamic re-determinization at player decision points in a given tree search. While MO-ISMCTS uses a single determinization throughout a simulation, RIS-MCTS re-samples a new valid hand for each player when it becomes their turn to act, maintaining consistency with their current information set. This re-sampling specifically addresses Hanabi's hidden-hand mechanic - players make decisions without knowing their own cards. The approach preserves the potential advantages of separate player trees for capturing broader strategic patterns while resolving the key limitation of MO-ISMCTS in Hanabi's unique information environment.

While RIS-MCTS can capture long-term strategic patterns through its separate trees, it still cannot fully address non-locality in terms of implicit communication. Unlike human players who might infer deeper meaning from a hint (such as its timing or context), the algorithm processes hints at face value. However, through its tree structure, it can learn to give hints that set up future plays, even if it cannot understand the implicit meanings that human players might derive from such actions.

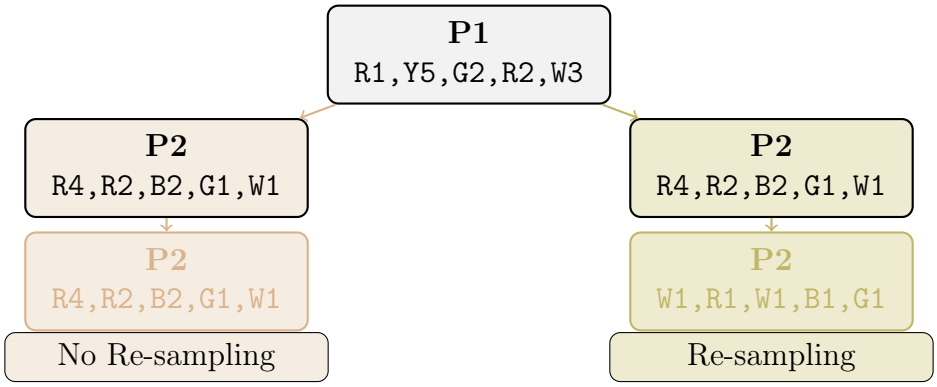

Figure 2.3: Comparison of sampling strategies. The left branch demonstrates traditional MO-ISMCTS where P2's hand remains constant. The right branch shows RIS-MCTS's re-sampling mechanism, where P2's hand is re-sampled at each decision point.

2.3 Improving Policies via Search in Cooperative Partially Observable Games

[Lerer et al., 2019] introduce SPARTA (Search for Partially Observing Teams of Agents), a novel approach to enhance existing policies in cooperative partially observable games. Rather than developing a new algorithm from scratch, SPARTA uses MCTS that acts as a policy improvement operator, it can be applied to any pre-existing strategy. The authors theoretically prove that SPARTA cannot decrease the expected reward relative to the blueprint policy, except for a bounded approximation error that decreases with more Monte Carlo rollouts.

2.4 How to Make the Perfect Fireworks Display: Two Strategies for Hanabi

The state-of-the-art approach in Hanabi draws from strategies developed for hat guessing games [Cox et al., 2015], a class of mathematical puzzles that emphasize information sharing through coded messages. To understand this connection, let's examine a simple hat guessing game that illustrates the core principles.

Consider five players seated around a table. Each wears either a red or blue hat, and while players can see everyone else's hat color, they cannot see their own. Starting with the youngest, each player must guess their own hat's color. With random guessing, the expected score (number of correct guesses) would be 2.5/5.

A more sophisticated approach emerges when the first player encodes information through their guess: they choose blue when observing an even number of blue hats, and red otherwise. To demonstrate this strategy's effectiveness, consider this configuration:

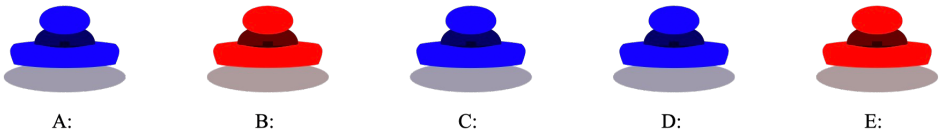

Figure 2.4: Hat configuration for the guessing game example

In this scenario, Player A sees three blue hats (odd number) and guesses red, which proves correct. Player B, observing two blue hats and knowing A's guess encoded parity information, deduces their hat must be red. Subsequent players make similar logical deductions based on previous guesses and visible hats.

This coordinated strategy achieves a perfect score of 5/5 in this case, and remarkably maintains an expected score of 4.5/5 across all possible hat configurations.

Methods

3.1 Environment

Given the computational demands of Monte Carlo Tree Search, selecting an efficient simulation environment is crucial for experimental success. We evaluated two prominent Hanabi implementations: Google DeepMind's environment [DeepMind, 2019] and PettingZoo's implementation [Brockman et al., 2016]. Our performance analysis was conducted on a MacOS Sonoma 14.6.1 system equipped with a 2.3 GHz Intel Core i9 (8 cores) processor and 16 GB of 2667 MHz DDR4 RAM.

Action Type	**Execution Time (seconds)**	
	PettingZoo	**DeepMind**
Discard	$(3.6 \pm 0.9) \times 10^{-4}$	$(1.4 \pm 1.3) \times 10^{-6}$
Play	$(2.1 \pm 1.7) \times 10^{-4}$	$(1.4 \pm 3.0) \times 10^{-6}$
Reveal Color	$(3.2 \pm 0.7) \times 10^{-4}$	$(1.1 \pm 1.2) \times 10^{-6}$
Reveal Rank	$(3.2 \pm 0.6) \times 10^{-4}$	$(1.1 \pm 1.1) \times 10^{-6}$

Table 3.1: Action execution times (in seconds) for different Hanabi environments, measured over 10^6 episodes.

Our benchmark results reveal a significant performance gap between the two implementations. The DeepMind environment demonstrates superior efficiency, executing actions at least 100 times faster than PettingZoo across all action types.

3.2 MCTS Baseline

In our analysis of Monte Carlo Tree Search applications to Hanabi, we identified RIS-MCTS [Goodman, 2019] as the most promising baseline for our AlphaZero augmentation. This implementation stands out among existing approaches for its effective handling of partial observability in cooperative games. We discuss this more later but it is important to note that we've never been able to reproduce the claimed results in the paper.

3.3 Optimizations

The computational efficiency of our baseline implementation was crucial for its eventual integration with AlphaZero. Our initial evaluation of the original implementation from [Goodman, 2019] revealed significant performance limitations, requiring 43 minutes to complete 100 episodes with 50 rollouts in a two-player setting.

Following [Chaslot et al., 2008], we implemented root parallelization, which involves conducting multiple independent tree searches simultaneously and aggregating their results. This approach not only provides superior performance compared to sequential execution but also yields better results. This optimization, combined with modifications to the DeepMind implementation, reduced the execution time to 4 minutes.

Further optimizations focused on three key areas: data structure efficiency, algorithmic improvements, and selective conversion of critical components to C/C++. These additional enhancements yielded a final execution time of just 2 minutes, representing a substantial 95% reduction from the original implementation.

3.4 AlphaZero

Following [Silver et al., 2017], we developed an AlphaZero implementation for Hanabi. Due to the complexity of adapting AlphaZero to a partially observable cooperative game, we adopted an incremental approach to understand the system's behavior.

Phase 1: Policy-Only

In our initial approach, we used only the policy head with MCTS. The action selection during search follows:

$$a_t = \underset{a}{\operatorname{argmax}}\, Q_{\text{env}}(s_t, a) + U(s_t, a) \tag{3.1}$$

where $Q_{\text{env}}(s_t, a)$ represents the average rewards received directly from environment simulations during MCTS. The exploration term $U(s, a)$ is defined as:

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \tag{3.2}$$

The policy network is trained to minimize:

$$\mathcal{L}_{\text{policy}} = -\boldsymbol{\pi}^T \log(\mathbf{p}) + c\|\theta\|_2^2 \tag{3.3}$$

where $\boldsymbol{\pi} \in \mathbb{R}^{|A|}$ represents the improved policy vector from MCTS, $\mathbf{p} \in \mathbb{R}^{|A|}$ is the network's output probability vector over the action space A, and $c\|\theta\|_2^2$ is an L2 regularization term with coefficient $c = 10^{-4}$.

Phase 2: Value Head Integration

Finally, we integrated the value head. The algorithm follows the same PUCT formula, but now $Q_\theta(s_t, a)$ is computed using the neural network, meaning it is the average rewards received from the network's predictions.

The complete loss function for Phase 3 becomes:

$$\mathcal{L}_{\text{total}} = \|v_\theta(s) - z\|_2^2 - \boldsymbol{\pi}^T \log(\mathbf{p}) + c\|\theta\|_2^2 \tag{3.4}$$

where z is the observed return from the game outcome, and $v_\theta(s)$ is the value network's prediction for state s.

Phase 3: Pretraining

To improve initial performance, we attempted pretraining both the policy and value heads of the network using data generated from our MCTS baseline.

Results

4.1 MCTS Baseline Evaluation

The MCTS implementation uses domain-specific heuristics to reduce the action space, limiting the branching factor through rules derived from expert gameplay strategies (see Section 6.2) and a simulation policy using a state-of-the-art rule-based agent.

Simulation Steps

We investigated how the number of simulation steps affects the MCTS performance.

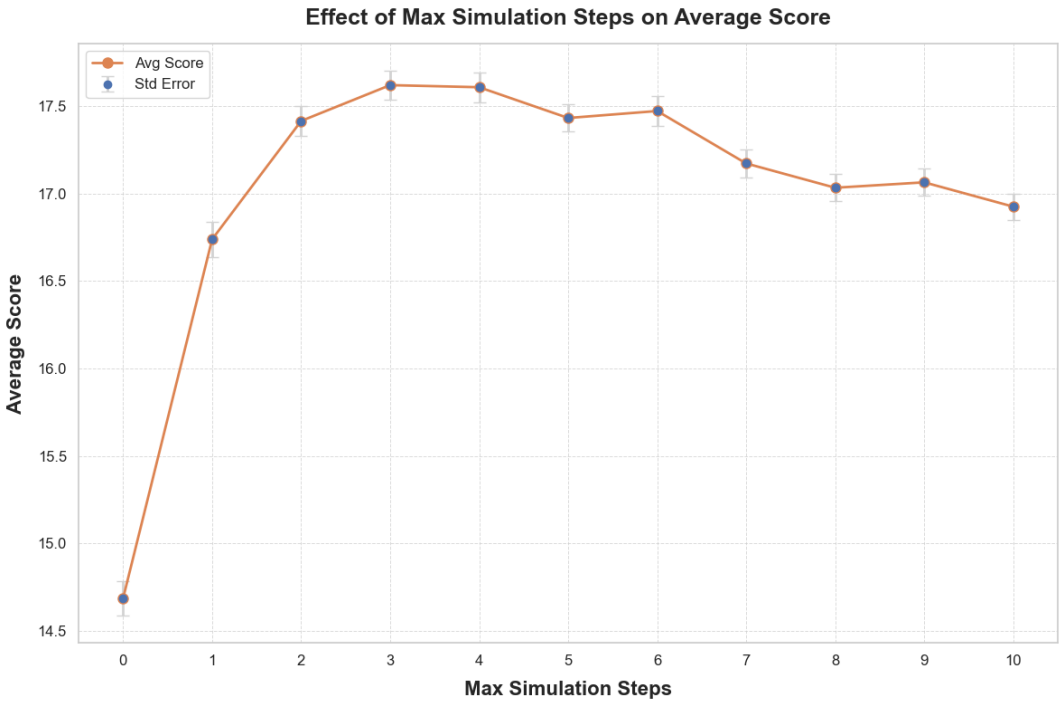

Figure 4.1: Effect of maximum simulation steps on average game score.

Figure 4.1 shows that performance increases sharply with the first simulation step and peaks at 3 steps with an average score of 17.62/25 points. Beyond this optimum, the performance gradually declines, suggesting that while the rule-based agent significantly improves initial decision-making, longer simulations may introduce noise due to re-determinizations.

For subsequent experiments, we chose to directly use the reward without simulation steps to avoid biasing our results.

Search Depth

We investigated the impact of MCTS maximum search depth on performance.

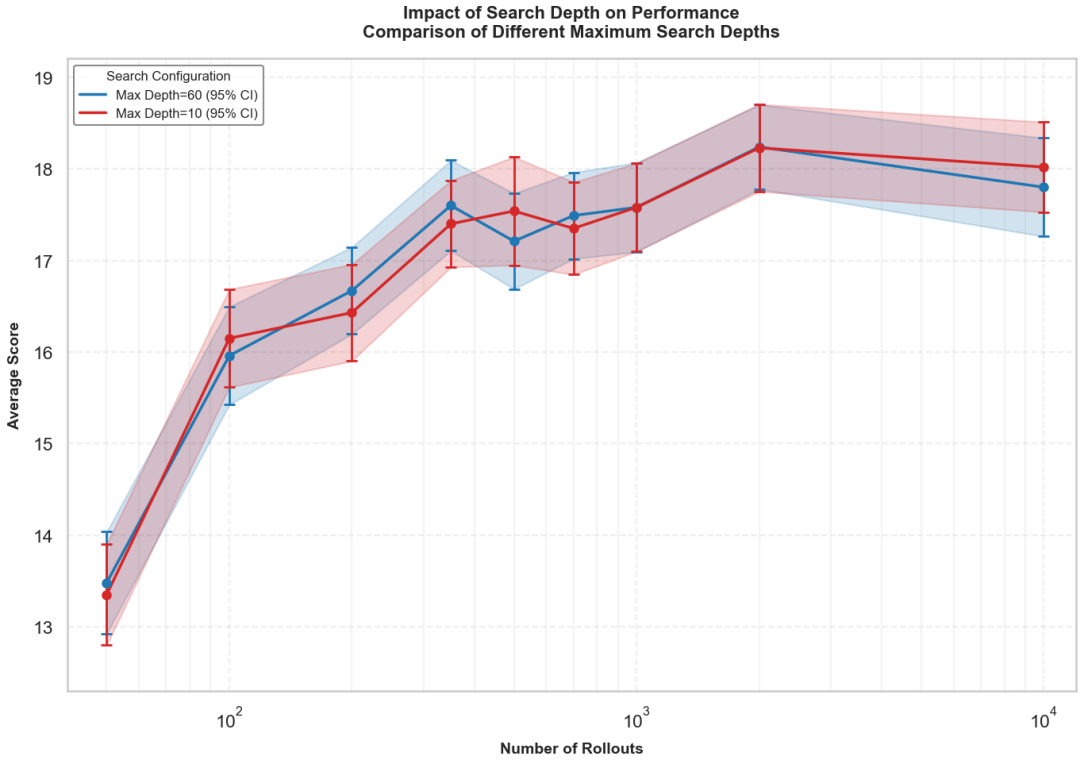

Figure 4.2: Effect of maximum search depth on average game score.

Figure 4.2 reveals no significant performance difference between search depths of 10 and 60, indicating that this parameter has negligible impact on the algorithm's effectiveness.

Impact of Domain Rules

Lastly, we evaluate the effectiveness of domain-specific rules constraining the search space (Section 6.2).

Figure 4.3 demonstrates the substantial impact of these rules on MCTS efficiency. The constrained version achieves an average score of 13.35/25 with just 50 rollouts, nearly matching the unconstrained algorithm's score of 14.72/25 achieved with 10^4 rollouts—a 200-fold difference in computational requirements. Both variants exhibit the expected MCTS behavior of improved performance with increased rollouts, confirming the algorithm's fundamental scaling properties.

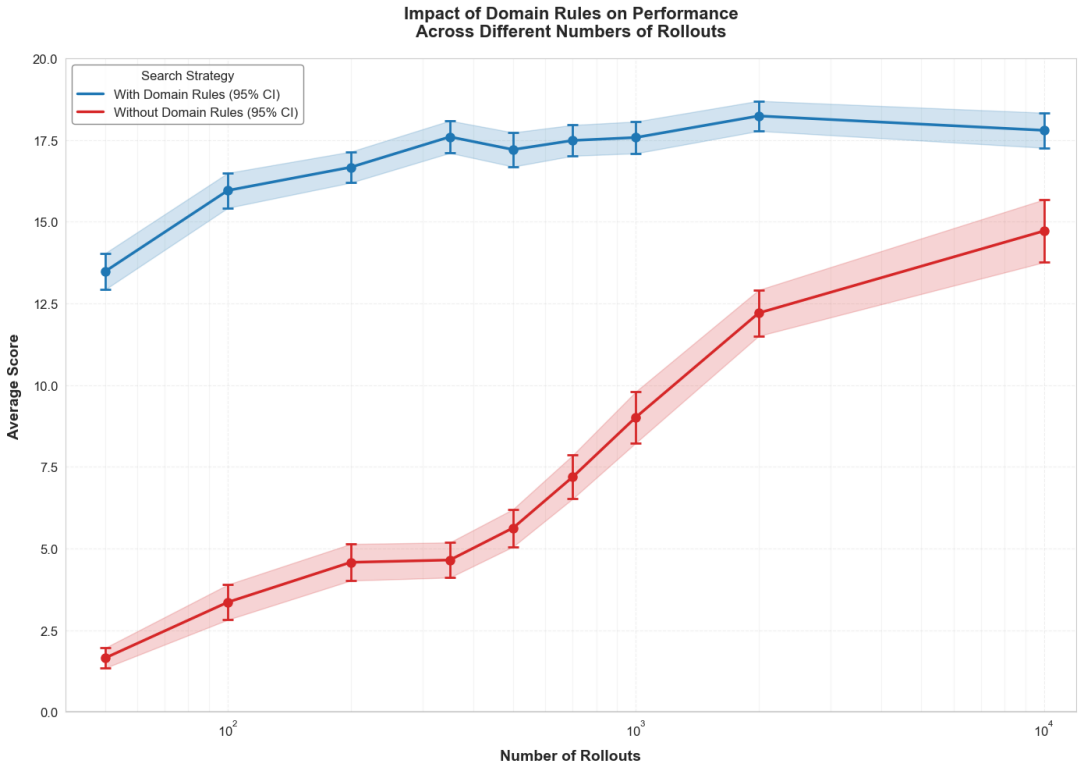

Figure 4.3: Performance comparison between MCTS with and without domain rules across different numbers of rollouts (shown with 95% confidence intervals).

4.2 AlphaZero

Network Architecture

We implemented a network with a shared backbone followed by separate policy and value heads. The backbone consists of three fully-connected layers [512, 256, 256] with GELU activations. The policy head uses two fully-connected layers with the final layer outputting logits for each action, while the value head uses two layers with a tanh activation to bound values between -1 and 1. All weights are initialized using He initialization ([He et al., 2015]).

Component	Architecture	Activation
Backbone	FC(input $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 256)	GELU
Policy Head	FC(256 $\rightarrow$ 256 $\rightarrow$ $num\_actions$)	GELU
Value Head	FC(256 $\rightarrow$ 128 $\rightarrow$ 1)	GELU + Tanh

Table 4.1: Neural network architecture details.

For more details concerning the input or the action space please refer to 6.3.

Training Configuration

We used prioritized experience replay [Schaul et al., 2016] with priority exponent $\alpha = 0.7$ and importance sampling $\beta = 0.4$. Training was performed with batch size 128 and

learning rate 10^{-4}. For the policy-only phase, we used the same architecture without the value head.

Policy-Only Results

Our attempt to directly implement AlphaZero on our baseline resulted in significantly decreased performance. We therefore adopted a stepwise approach to better understand the system dynamics. We first implemented a policy network without the value head to guide MCTS toward promising states. This modification showed minimal improvement over the baseline and no learning progression during training, though it validated the basic implementation. However, detailed analysis of gameplay revealed fundamental limitations in the baseline implementation.

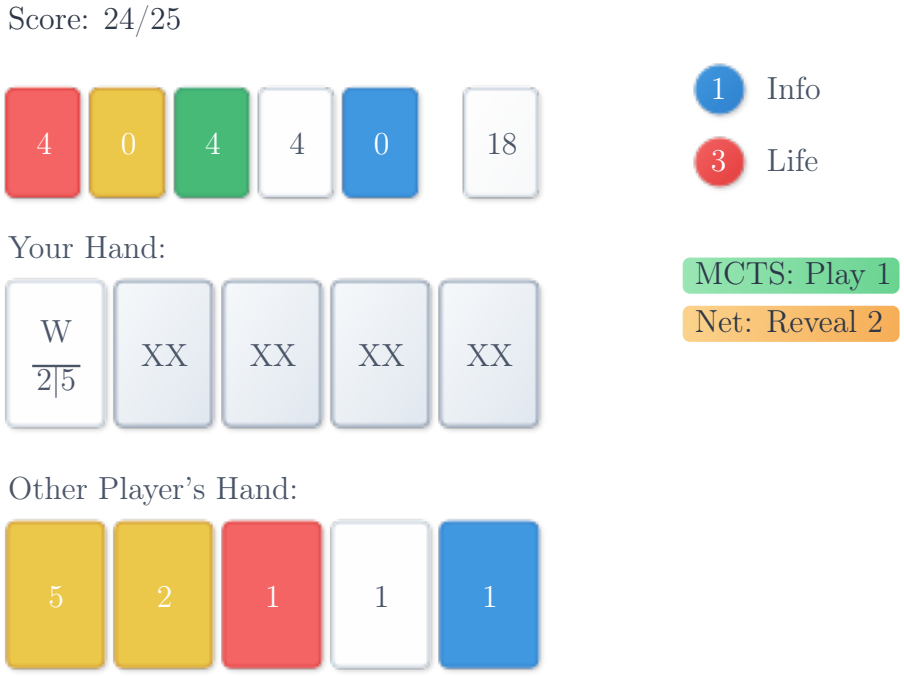

Figure 4.4: Game state visualization showing a critical decision point. The first card is either a White 2 or 5.

Figure 4.4 illustrates a critical decision point where the network suggests revealing information while MCTS recommends playing the first card. While playing a White 5 would be advantageous, an incorrect play wastes both a life token and the accumulated information about that card. Human players typically either wait for complete information or discard the card if no new information is forthcoming.

This issue becomes more pronounced in Figure 4.5, where the player attempts to play a card that could be any color five—a move no experienced player would consider due to its excessive risk. Though both plays succeeded by chance, this behavior stems from the reward system's structure. When determinization identifies a playable card, it generates immediate positive value that biases MCTS toward risky plays. While we could theoretically penalize value-losing paths, implementing such penalties proves challenging as any path could potentially result in point loss.

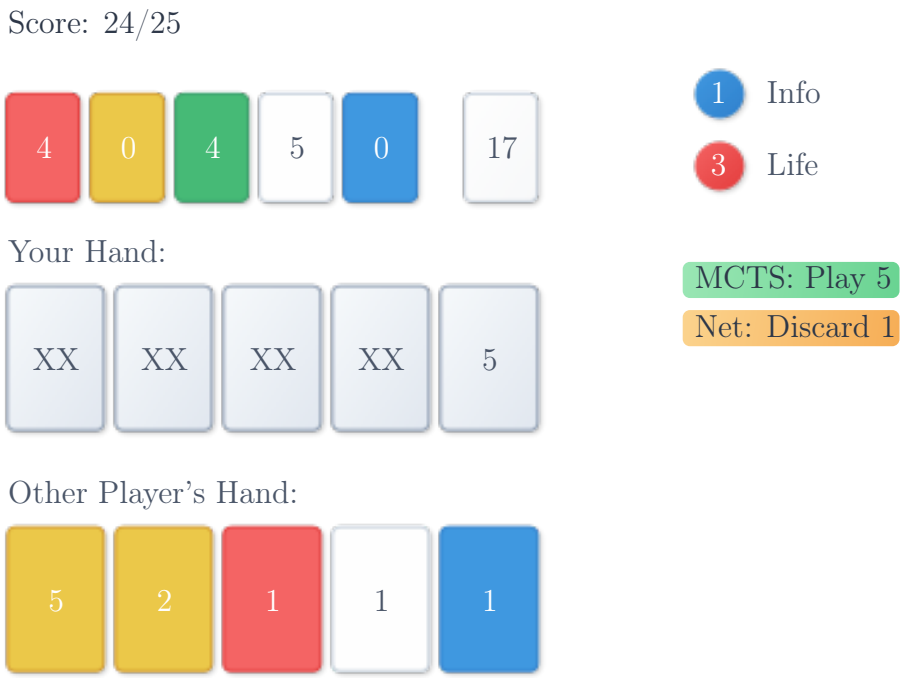

Figure 4.5: Game state visualization showing a critical decision point. The last card is a 5 but we don't know the associated color.

Standard AlphaZero Results

To address these limitations, we hypothesized that incorporating a value head into the network could better capture reward patterns from the input space. Figure 4.6 shows the training progress with 100 rollouts per move. The algorithm starts with an initial average score of 8.95/25 and demonstrates steady improvement over 1400 episodes, reaching 13.80/25. However, this remains below our baseline performance of 15.96/25—a gap of approximately two points. While the consistent upward trend in performance suggests potential for further improvement, hardware constraints prevented us from extending the training period to test this hypothesis.

Pretraining Results

Our final approach explored pretraining a policy-value network using high-quality data generated from our baseline implementation. We collected data from 1,000 episodes with an average score of 17.24/25. However, the network's ability to exploit this data fell short of expectations, exhibiting learning patterns similar to those observed in standard training. We hypothesize that this limitation stems from the network's difficulty in accurately generating value estimates across the vast input space. In our two-player setting, each game state is represented by a binary vector of at least 658 bits, resulting in approximately $2^{658} \approx 1.2 \times 10^{198}$ possible states. Given this astronomical state space, it is probable that during actual gameplay, the network encountered states with minimal similarity to those in the training dataset, limiting the effectiveness of the pretraining approach.

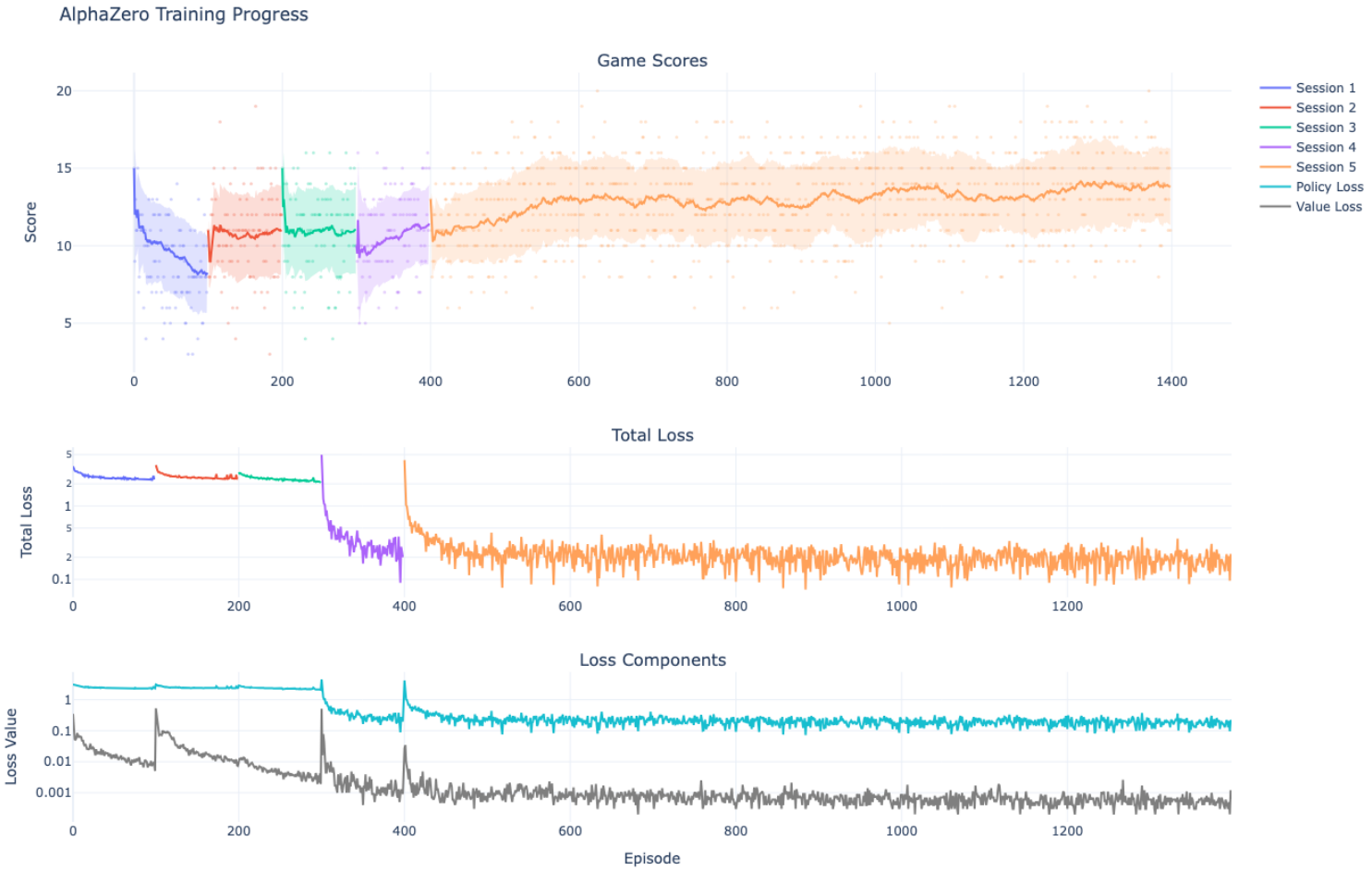

Figure 4.6: AlphaZero training progression showing game scores (top), total loss (middle), and loss components (bottom) over 1400 episodes. Game scores represent 50-episode moving averages, with shaded areas indicating score variance across five training sessions. Policy and value loss components are plotted on a logarithmic scale.

Conclusion

Our investigation into applying AlphaZero to Hanabi illuminates significant challenges in adapting modern reinforcement learning methods to partially observable cooperative games. While our baseline MCTS implementation achieved competitive performance, it revealed fundamental limitations in the reward structure that inadvertently encouraged high-risk plays. The algorithm's tendency to attempt potentially catastrophic moves based on favorable determinizations highlights a critical flaw in the current approach to handling partial observability.

The implementation of AlphaZero, despite showing consistent learning progression, failed to surpass our MCTS baseline. This shortfall can be attributed to several factors, most notably the astronomical state space of Hanabi. The sheer magnitude of this space poses a fundamental challenge for neural network generalization, particularly in capturing the nuanced patterns of information sharing that characterize expert human play.

Furthermore, our approach to handling hidden information through determinization, while computationally efficient, may be fundamentally limiting the algorithm's ability to develop sophisticated communication strategies.

Looking forward, several critical research directions emerge from our findings. The development of more sophisticated network architectures specifically designed for partial observability could potentially bridge the current performance gap. Additionally, the exploration of alternative approaches to hidden information handling, perhaps drawing inspiration from recent advances in transformer architectures and attention mechanisms, might better capture the game's inherent complexity. The investigation of explicit communication protocols between agents, possibly incorporating ideas from emergent communication in multi-agent systems, could also prove fruitful.

Our work ultimately demonstrates that while the direct application of AlphaZero to Hanabi faces significant challenges, the observed learning progression suggests potential for success with substantial architectural and algorithmic innovations. The considerable gap between current performance and theoretical optimum indicates that this remains a rich area for future research in reinforcement learning for cooperative games.

Bibliography

Jean-Francois Baffier, Man-Kwun Chiu, Yago Diez, Matias Korman, Valia Mitsou, André van Renssen, Marcel Roeloffzen, and Yushi Uno. Hanabi is np-hard, even for cheaters who look at their cards, 2017. URL https://arxiv.org/abs/1603.01911.

Nolan Bard, Jakob N. Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H. Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, Iain Dunning, Shibl Mourad, Hugo Larochelle, Marc G. Bellemare, and Michael Bowling. The hanabi challenge: A new frontier for ai research. *Artificial Intelligence*, 280:103216, March 2020. ISSN 0004-3702. doi: 10.1016/j.artint.2019.103216. URL http://dx.doi.org/10.1016/j.artint.2019.103216.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. URL http://arxiv.org/abs/1606.01540.

Guillaume M. J. B. Chaslot, Mark H. M. Winands, and H. Jaap van den Herik. Parallel monte-carlo tree search. In H. Jaap van den Herik, Xinhe Xu, Zongmin Ma, and Mark H. M. Winands, editors, *Computers and Games*, pages 60–71, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. ISBN 978-3-540-87608-3.

Peter Cowling, Edward Powley, and Daniel Whitehouse. Information set monte carlo tree search. *IEEE Transactions on Computational Intelligence and Ai in Games*, 4: 120–143, 06 2012. doi: 10.1109/TCIAIG.2012.2200894.

Christopher Cox, Jessica De Silva, Philip Deorsey, Franklin Kenter, Troy Retter, and Josh Tobin. How to make the perfect fireworks display: Two strategies for hanabi. *Mathematics Magazine*, 88:323, 12 2015. doi: 10.4169/math.mag.88.5.323.

DeepMind. The Hanabi learning environment. https://github.com/deepmind/hanabi-learning-environment, 2019. [Online; accessed 1-February-2019].

James Goodman. Re-determinizing information set monte carlo tree search in hanabi, 2019. URL https://arxiv.org/abs/1902.06075.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015. URL https://arxiv.org/abs/1502.01852.

Adam Lerer, Hengyuan Hu, Jakob Foerster, and Noam Brown. Improving policies via search in cooperative partially observable games, 2019. URL https://arxiv.org/abs/1912.02318.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay, 2016. URL https://arxiv.org/abs/1511.05952.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL https://arxiv.org/abs/1712.01815.

Annexes

6.1 AlphaZero Policy Observations

During our experiments, we investigated whether our AlphaZero network was producing highly biased probabilities, potentially assigning most of the weight to a single action, which is not expected. Below are two examples, though many more can be found in the repository. Our overall observation is that the network's output policies are well-distributed, with the probability mass evenly spread among available actions. In contrast, MCTS tends to heavily favor one action, which aligns with the expected behavior of MCTS.

Action	**MCTS**	**Network**	**Selected**
DISCARD card 4	0.008	0.350	–
Reveal Y to P1	0.005	0.278	–
Reveal 1 to P1	0.987	0.372	$\star$

Table 6.1: Standard AlphaZero prior probabilities and after a few rollouts of MCTS

Action	**MCTS**	**Network**	**Selected**
DISCARD card 4	0.010	0.576	N
PLAY card 2	0.990	0.424	M

Table 6.2: Policy-only AlphaZero prior probabilities and after a few rollouts of MCTS

6.2 Rules Reducing the Branching Factor of MCTS Baseline

These rules are heuristics used to reduce the branching factor from 32 to 9, meaning no other actions are investigated except the ones below, see [Goodman, 2019] if needed.

1. **Information-Maximizing Hints**: Provides hints that reveal most new information.
2. **Playable Card Hints**: Reveals either the color or rank of a playable card in other players' hands.
3. **Dispensable Card Information**: When information tokens are abundant, reveals information about cards that are discardable.
4. **Complete Playable Information**: Completes partial information about playable cards by revealing the missing attribute (color or rank).

5. **Complete Dispensable Information**: Completes partial information about dispensable cards.
6. **Complete Unplayable Information**: Reveals missing information about cards that are neither immediately playable nor discardable.
7. **Safe Play**: Plays a card if the probability of it being playable exceeds 70%.
8. **Late Game Safe Play**: Adopts a more aggressive playing strategy in the late game, playing cards with at least 40% probability of being playable when 5 or fewer cards remain.
9. **Confident Discard**: Discards the card that the player is most confident is discardable.

6.3 Environment Details

Observation Space

Index Range	Description
0–124	Other Player's Hand (5 cards × 25 bits each)
125–126	Missing Card Indicators (1 bit per player)
127–166	Deck Size (Thermometer Encoding)
167–171	Red Firework Stack
172–176	Yellow Firework Stack
177–181	Green Firework Stack
182–186	White Firework Stack
187–191	Blue Firework Stack
192–199	Information Tokens (Thermometer)
200–202	Life Tokens (Thermometer)
203–252	Discard Pile
253–254	Last Player ID
255–258	Last Move Type
259–260	Last Move Target Player
261–265	Last Move Color Revealed
266–270	Last Move Rank Revealed
271–275	Last Move Outcome
276–280	Last Move Position
281–305	Last Move Card
306–307	Last Move Success
308–657	Card Knowledge (25+5+5 bits × 5 cards × 2 players)

Table 6.3: Observation Vector Encoding for 2-Player Hanabi

Action Space

Action ID	Action Type
Discard Actions	
0	Discard card from position 0
1	Discard card from position 1
2	Discard card from position 2
3	Discard card from position 3
4	Discard card from position 4
Play Actions	
5	Play card from position 0
6	Play card from position 1
7	Play card from position 2
8	Play card from position 3
9	Play card from position 4
Color Reveal Actions	
10	Reveal Red cards for Player 1
11	Reveal Yellow cards for Player 1
12	Reveal Green cards for Player 1
13	Reveal White cards for Player 1
14	Reveal Blue cards for Player 1
Rank Reveal Actions	
15	Reveal Rank 1 cards for Player 1
16	Reveal Rank 2 cards for Player 1
17	Reveal Rank 3 cards for Player 1
18	Reveal Rank 4 cards for Player 1
19	Reveal Rank 5 cards for Player 1

Table 6.4: Action IDs and their corresponding actions in Hanabi

6.4 Rules of Hanabi

Hanabi is a cooperative card game where players work together to create a spectacular fireworks display by playing cards in a specific sequence. The game is unique because players cannot see their own cards but can see the cards of their teammates. Communication is limited, and players must give clues to help each other play their cards correctly. The objective is to play cards in ascending order of each color, from 1 to 5. Here are the detailed rules of Hanabi :

Game Material

50 fireworks cards in five colors (red, yellow, green, blue, white) :

- 10 **cards per color** with the values 1, 1, 1, 2, 2, 3, 3, 4, 4, 5
- 8 **clock** tokens
- 3 **fuse** tokens.

Aim of the Game

Hanabi is a cooperative game, meaning all players play together as a team. The players have to play the fireworks cards sorted by colors and numbers. However, they cannot see their own hand cards, and so everyone needs the advice of their fellow players. The more cards the players play correctly, the more points they receive when the game ends.

The Game

The player which is dressed in the most colorful way is appointed first player and sets the tokens in the play area. The eight **clock** tokens are placed in a box. The three **fuse** tokens are stacked up on the table.

Now the fireworks cards are shuffled. Depending on the number of players involved, each player receives the following hand :

- With 2 or 3 players : 5 cards in hand
- With 4 or 5 players : 4 cards in hand

Important : Unlike other card games, players may not see their own hand! The players take their hand cards so that the back is facing the player. The fronts can only be seen by the other players. The remaining cards are placed face down in the draw pile in the middle of the table. The first player starts.

Game Play

Play proceeds clockwise. On a player's turn, they must perform exactly one of the following :

1. Give a hint
2. Discard a card
3. Play a card.

The player has to choose an action. A player may not pass!
Important : Players are not allowed to give hints or suggestions on other players' turns!

A. Give a hint To give a hint one **clock** token must be removed from the box. If there are no **clock** tokens in the box then a player may not choose the **Give a hint** action. Now the player gives a teammate a hint. They have one of two options :

1. **Color Hint.** The player chooses a color and indicates to their teammate which of their hand cards match the chosen color by pointing at the cards. **Important :** The player must indicate all cards of that color in their teammate's hand! Example : "You have two yellow cards, here and here." Indicating that a player has no cards of a particular color is not allowed!
2. **Value Hint.** The player chooses a number value and gives a teammate a hint in the exact same fashion as a Color Hint. Example : "You have a 5, here."

B. Discard a card To discard a card one **clock** token must be returned to the box. If there are no **clock** outside the box then a player may not choose the **Discard a card** action. Now the player discards one card from their hand (without looking at the fronts of their hand cards) and discards it face-up in the discard pile near the draw deck. The player then draws another card into their hand in the same fashion as their original hand cards, never looking at the front.

C. Play a card By playing out cards the fireworks are created in the middle of the table. The player takes one card from their hand and places it face up in the middle of the table. Two things can happen :

1. **The card can be played correctly.** The player places the card face up so that it extends a current firework or starts a new firework.
2. **The card cannot be played correctly.** The fuse is getting shorter and time is running out. The player removes a **fuse** token from the stack. The incorrect card is discarded to the discard pile near the draw deck.

In either case, the player then draws another card into their hand in the same fashion as their original hand cards, never looking at the front.

The Fireworks

The fireworks will be in the middle of the table and are designed in five different colors. For each color an ascending series with numerical values from 1 to 5 is formed. A firework must start with the number 1 and each card played to a firework must increment the previously played card by one. A firework may not contain more than one card of each value.

Bonus

When a player completes a firework by correctly playing a 5 card then the players receive a bonus. One **clock** token is returned to the box. If all tokens are already white-side-up then no bonus is received. Play then passes to the next player (clockwise).

Ending the Game

The game can end in three ways :

1. The third **fuse** token is removed from the stack (thus revealing the explosion). The game ends immediately, and the players earn zero points.
2. The players complete all five fireworks correctly. The game ends immediately, and the players celebrate their spectacular victory with the maximum score of 25 points.
3. If a player draws the last card from the draw deck, the game is almost over. Each player—including the player who drew the last card—gets one last turn.

Finally, the fireworks will be counted. For this, each firework earns the players a score equal to the highest value card in its color series. The quality of the fireworks display according to the rating scale of the "International Association of Pyrotechnics" is :

- **0–5** : Oh dear! The crowd booed.
- **6–10** : Poor! Hardly applauded.
- **11–15** : OK! The viewers have seen better.
- **16–20** : Good! The audience is pleased.
- **21–24** : Very good! The audience is enthusiastic!
- **25** : Legendary! The audience will never forget this show!

Important Notes and Tips

- Players may rearrange their hand cards and change their orientation to help themselves remember the information they received. Players may not ever look at the front of their own cards until they play them.
- The discard pile may always be searched for information.
- Hanabi is based on communication—and non-communication—between the players. If one interprets the rules strictly then players may not, except for the announcements of the current player, talk to each other. Ultimately, each group should decide by its own measure what communication is permitted. Play so that you have fun!